Labo 2: Max detection in array
Geavanceerde computerarchitecturen

Jan Martens Leon Cherchye

November 15, 2024



Session Date: 8 november, 2024

Class: Ma ELICT optie software-engineering

1 Introduction

In dit labo vergeleken we verschillende manieren om de maximum waarde in een array te ontdekken. We bekeken de standaard-manier gebruikmakende van de cpu door alle elementen in de array te overlopen, alsook twee verschillende manieren gebruikmakende van de GPU.

2 Resultaten

De prestaties van de CPU en GPU zijn afhankelijk van de grootte van de arrays. Voor kleinere arrays, tot ongeveer 50.000 elementen, behaalt de CPU betere resultaten. Echter, zodra de arraygrootte een bepaalde drempel overschrijdt, presteert de GPU aanzienlijk beter. Een weergave op logaritmische schaal die dit effect weergeeft, is te vinden in Figure 1.
We kunnen eenvoudig controleren of de resultaten kloppen door deze te vergelijken met de resultaten van de CPU. De CPU-methode werkt gegarandeerd correct, aangezien we element per element de hele array overlopen.

3 Input groter dan 2048 elementen

Voor input groter dan 2048 blijft de atomicMax-operatie correct werken vanwege de ingebouwde thread-veiligheid, maar voor grote arrays kan dit prestatieproblemen opleveren door verhoogde concurrentie tussen threads. Een efficiëntere aanpak is een hiërarchische reductie: eerst berekent elke blok zijn deelmaximum, waarna een tweede kernel-uitvoering het globale maximum bepaalt uit deze deelresultaten. Deze aanpak vermindert de belasting op één variabele, dus de concurrentie daalt, en verhoogt de efficiëntie bij grote datasets.
Het implementeren van CUDA-streams kan de prestaties verder verbeteren door data in parallelle delen te verwerken.

4 Verwerk alle elementen met minder threads dan elementen

Om op deze manier toch tot een correct resultaat te komen, zouden we ervoor moeten zorgen dat elke thread meerdere data-elementen bekijkt.
Elke thread kijkt dus naar een aantal elementen in de shared array, berekent daar het maximum van en wacht dan op de volgende threads tot die van hun ge-assigende elementen het max hebben berekent. Als alle threads klaar zijn, doen we de reductie binnen elke block en kan de volgende itteratie beginnen.
Dit zou sowieso resulteren in een tragere berekening.

5 2D array

Bij het verwerken van een 2D-array op de GPU wordt de matrix lineair in het geheugen geplaatst, omdat GPU-geheugen efficiënter omgaat met een enkelvoudige (1D) arraystructuur. Met 2D-thread- en block-indexen kan elk element in de array snel worden benaderd door de juiste rij- en kolomindex om te zetten naar een enkele index. Een voorbeeld van de syntax voor een 2D-array van width x height is:

```
int row = blockIdx.y * blockDim.y + threadIdx.y;
int col = blockIdx.x * blockDim.x + threadIdx.x;
int index = row * width + col;

if (row < height && col < width) {
    atomicMax(maxVal, array[index]);
}
```

6 Grafieken

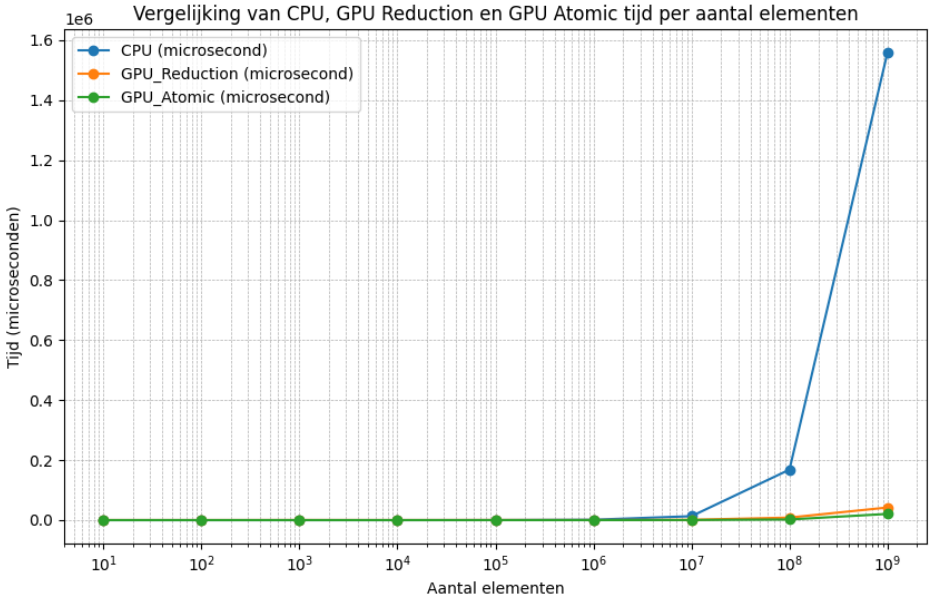


Figure 1: Grootte Array - time ellapsed

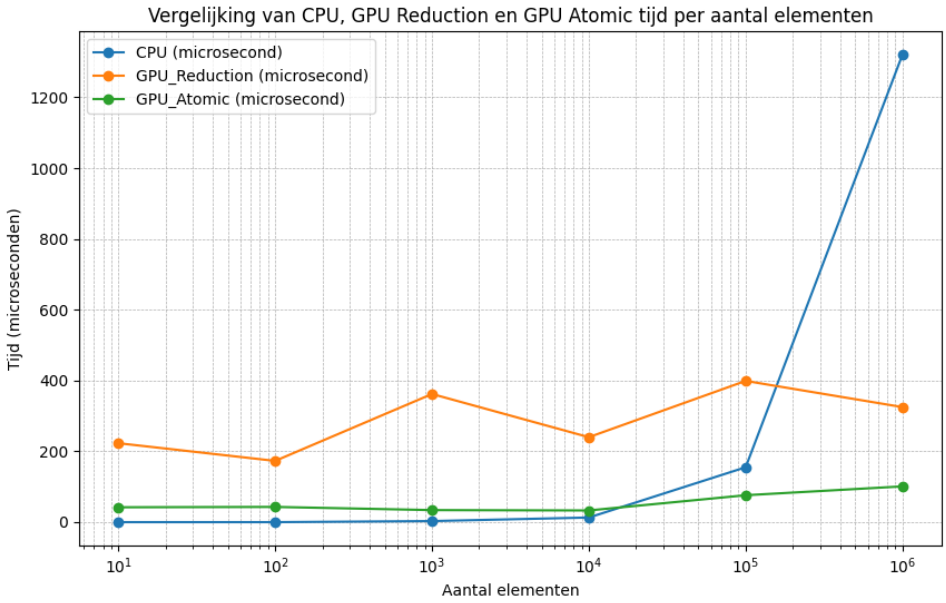


Figure 2: Grootte Array - time ellapsed, zoomed in

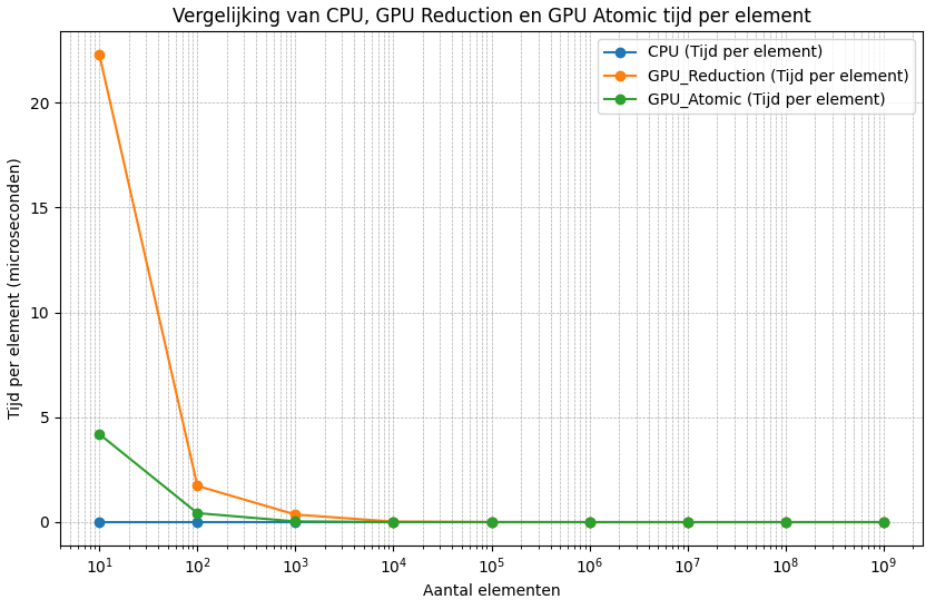


Figure 3: Grootte Array - processing time per element

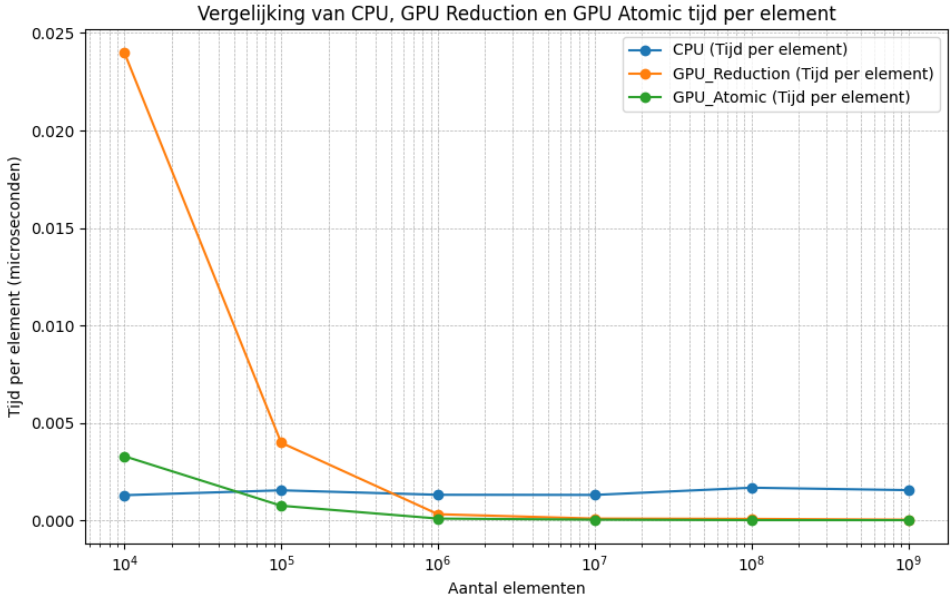


Figure 4: Grootte Array - processing time per element, zoomed in